

目录（我只是目录，点击不会跳转的那种）

一. 分布式系统特征（共 6 题）

二. 系统模型（共 6 题）

三. 进程间通信（共 5 题）

四. 操作系统支持（共 5 题）

第 2 题答案无

五. 同步化（共 6 题）

第 3 题答案无

六. 分布式文件系统（共 2 题）

第 2 题答案无

七. 分布式系统前沿（共 2 题）



正文

一. 分布式系统特征（共 6 题）

1. 什么叫分布式系统？举例说明现实生活中的分布式系统，阐述系统组成部分与功能特点。

答：分布式系统是一个硬件或软件组件分布在不同的网络计算机上，通过消息传递进行通信和协调的系统。

举例：Web 搜索（Google，百度），网上购物（京东，淘宝），智能交通（地图服务-复杂系统），医疗保健（安全私密性）

举例：Web 搜索

1. 产品应用层：搜索，广告，电子邮件，地图，视频，聊天，博客。

2. 基础设施层：Google 的云计算平台：其 GFS 分布式文件系统，针对 Google 应用程序的特点提出的 Map/Reduce 编程模式，分布式的锁机制 Chubby 以及 Google 开发的模型简化的大规模分布式数据库 Big Table。

3. 计算平台层：分布在一堆不同的数据中心中一堆机器

举例 - Web 搜索

1. 底层物理设施，它由超大数目的位于全世界多个数据中心的联网计算机组成；

2. 分布式文件系统，支持超大文件，并根据搜索和其他应用的使用方式（特别是在文件中以快速而持久的速度读取）进行了深度优化；

3. 相关的结构化分布式存储系统，它提供对超大数据集的快速访问；锁服务，它提供诸如分布式加锁和协定等分布式系统功能；

4. 编程模式，它支持对底层物理基础设施上的超大并行和分布式计算的管理。

2. 分布式系统的特征是什么？

答：

并发性 - 程序通过共享资源并行

缺乏全局时钟 - 分布式系统中的计算机只能通过本地时钟交换消息；不可预测的消息延迟限制了准确性；没有一个全局时钟的概念。

故障独立性 - 计算机中的故障或程序中的异常马上不能被与之通讯的其他组件感知；系统设计者需要为故障处理做计划。

3. 给出能被共享的 5 种类型的硬件资源和 5 种类型的数据或软件资源。给出它们在实际的分布式系统中发生共享的例子。

答：

网络资源共享：服务器上的驱动器，文件，数据资源等能够被网络上的用户访问，使计算机网络中的数据互通有无，分工协作，从而提高硬件和数据的使用率

答：硬件：打印机，磁盘，扫描仪，光盘，CPU,内存

数据或软件资源：文件，搜索引擎，数据库，图片，应用软件，office

比如 WEB 上的搜索引擎给全世界的用户提供工具，用户间不需要直接接触

文件和数据的共享的方式：计算迁移和数据迁移

4. 在分布式系统中，常常说，“避免性能瓶颈”。可否举例说明并谈谈你的观点。

答：

算法应是分散型的避免性能瓶颈。如一个存储子系统由于不能以足够的速率提供数据去驱动网络适配器而引起瓶颈。当网上有多个很强的客户，其集中器上就需要有缓冲区能保存请求，提供智能交换和对服务器的高速访问。如许多用户访问同一个 WEB 页面，会引起网络性能下降。通过缓存和复制来提高频繁使用的资源性能。复制数据，缓存，部署多服务器以处理经常执行的任务从而使几个类似的任务能并发地完成。

5. 在故障处理（Failurehandling）中，什么叫容错？什么叫冗余？

答：

冗余：指重复配置系统的一些部件,当系统发生故障时,冗余配置的部件介入并承担故障部件的工作,由此减少系统的故障时间。通常指通过多重备份来增加系统的可靠性。

容错：容错是用冗余的资源使计算机具有容忍故障的能力，即在产生故障的情况下，仍有能力将指定的算法继续完成。

容错技术是指在一定程度上容忍故障的技术，也称为故障掩盖技术（fault masking）。采用容错技术的系统称容错系统。

容错主要依靠冗余设计来实现，它以增加资源的办法换取可靠性。由于资源的不同，冗余技术分为硬件冗余、软件冗余、时间冗余和信息冗余。

6. 现今分布式操作系统的挑战有：Heterogeneity（异构性），Openness（开放性），Security（安全性），Scalability（可伸缩性），Failurehandling（故障处理），Concurrency（并发性），Transparency（透明性），等。分别给出挑战的定义，举例与详细分析挑战涉及的关键技术。

答案来自百度

答：

一：异构性。网络、计算机硬件、操作系统、编程语言、由不同的开发者完成的软件实现都是造成异构性的主因。其中网络的异构通过互联网协议（TCP/IP 协议）相互通信而被屏蔽；中间件：屏蔽了底层网络、硬件、操作系统和编程语言的异构，它为分布式应用和服务器提供了一致的计算模型，包括 RPC、远程事件通知、远程 SQL 访问和分布式事物调用；虚拟机也是使代码到处运行的一种方法。

二：开放性。它取决于新的资源共享服务能被增加和供多种客户程序使用的程度。特征就是发布系统的关键接口，使其基于一致的通信机制，让不同提供商提供异构硬件和软件。

三：安全性。包括三个部分：机密性（防止泄露给未授权的个人）、完整性（防止被改变或被破坏）、可用性（防止对访问资源的手段的干扰）。其中有两个重要的安全问题：拒绝服务攻击和移动代码的安全性仍然没有得到圆满解决。

四：可伸缩性。顾名思义就是随着资源数量的增加和用户访问的增加，系统仍然能保持其有效性，该系统就被称为可伸缩的。其中有控制物理资源的开销、控制性能损失、防止软件资源用尽和避免性能瓶颈四大挑战。

五：故障处理。在一个分布式系统中，硬件或软件都会出现未知的故障或者不正常运行，因此故障处理是贯穿整个系统的难题。容错（设计容错机制如重传）、故障恢复（数据恢复或“回滚”保证一致性）、冗余（多条路由或者备份等技术）都是故障处理技术。

六：并发性。多个用户对同一资源的使用，要保持操作的正确性就必须在数据保持一致的基础上同步。如使用操作系统的信号量。

七：透明性。

1. 访问透明性：用相同的操作访问本地资源和远程资源（电子邮件）

2. 位置透明性：不需要知道资源的物理位置或网络位置（电子邮件、URL）
3. 并发透明性：几个进程能并发的使用共享资源而不互相干扰
4. 复制透明性：使用资源的多个实例提升可靠性和性能，而用户和程序员无需知道副本的相关信息
5. 故障透明性：屏蔽错误
6. 移动透明性：资源和客户能够在系统内移动而不受影响（移动电话）
7. 性能透明性：负载变化时，系统能够被重新配置以提高性能
8. 伸缩透明性：系统和应用能够进行扩展而不改变系统结构 and 应用算法。如 12306 购票系统，需要处理并行性；淘宝双 11，需要处理的是可伸缩性



二. 系统模型（共 6 题）

1. 分布式系统模型设计时，设计者常常面对的分布式系统的困难与威胁问题有哪些？

答案来自课件

答：

1. 使用模式的多样性：

系统组件承受各种工作负载

例如：Web 每天有几百万的访问量

系统断线或连接不稳定

例如：系统中包括移动计算机

系统对带宽与延迟的特殊要求

例如：多媒体应用

2. 系统环境的多样性：

容纳异构硬件、操作系统和网络

网络在性能上有很大的不同

例如：无线网的速度只达到局域网的几分之一

支持不同规模的系统

从几十台计算机到几百万台计算机

3. 内部问题：（并发引起的数据不一致，如购票系统）

非同步时钟

冲突的数据更新

系统组件的软硬件故障

4. 外部问题：（web 欺骗及防范技术：网银口令攻击）

数据完整性、保密性的攻击、服务拒绝攻击

2. 分布式系统体系结构元素包括：通信实体、通信范型、角色和责任、放置，以客户-服务器系统体系结构为例解释相应元素概念。

答案来自书《分布式系统：概念与设计（原书第 5 版）》

答：

通信实体：客户端与服务器端的通信（对象，组件，web 服务），底层进程间的通信。

通信范型：分布式系统中实体如何通信。客户-服务器结构中最常见的通信范型是远程调用，如用于支持客户-服务器计算的请求-应答协议是一个有效的模式。

角色和责任：在客户-服务器结构中，进程扮演服务器和客户的角色。特别是，为了访问服务器管理的共享资源，客户进程可以与不同主机上的服务器进行交互。

放置：对象或服务这样的实体是怎样映射到底层的物理分布式基础设施上的，物理分布式基础设施由大量的机器组成，这些机器通过一个任意复杂的网络互联。从决定分布式系统特性的角度而言，放置是关键的，这些特

性大多数与性能相关，也包括其他特性如可靠性和安全性。放置需要考虑实体间的通信模式、给定机器的可靠性和它们当前的负载、不同机器之间的通信质量等。

3. 什么叫层次化软件体系结构？举例说明。

答：

层次化体系结构与分层体系结构是互补的，层次化是一项组织给定层功能的技术。

如搜索引擎，金融决策支持系统（三层）：用户界面层，处理层，数据层

4. 什么叫瘦客户？举例说明你的观点。

答：

瘦客户是指一个软件层，它支持用户端的计算机上基于窗口的用户界面，而在远程的计算机上执行应用程序。

不足：潜在的长响应时间。



主要缺点：在交互频繁的图形活动，如 CAD 和图像处理中，因为网络和操作系统的延迟，用户感受到的延迟会因为瘦客户和应用进程之间传输图像和向量信息而增大到不可接受的程度

5. 举例说明并图示异步分布式系统中，不同网络结点间进程的“事件的实时排序”。（参考图 2-13）

答：

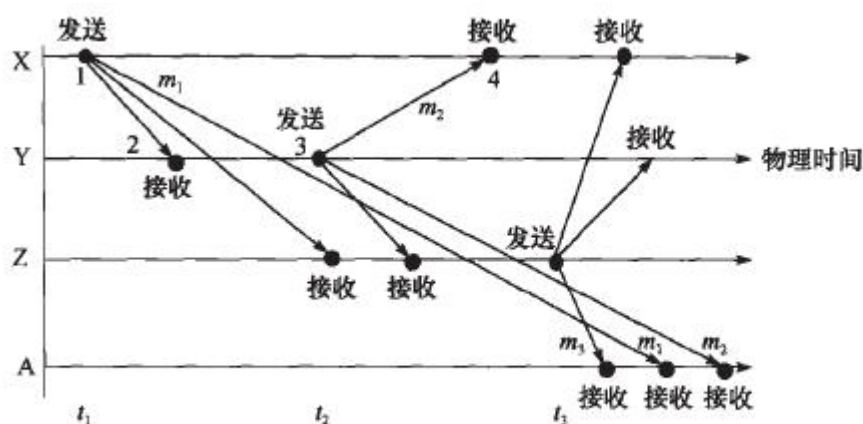


图 2-13 事件的实时排序

由于时钟不能精确同步，提出了逻辑时间的模型，因为逻辑上，我们知道消息在它发送之后才被接受，所以对上图所示的成对事件给出一个逻辑排序。如：X 在 Y 接受到 M_1 前发送 M_2 ，Y 在 X 接受到 M_2 前发送 M_2 。Y 在发送 M_2 前接收 M_1

6. 分布式系统的基础模型有哪些？分别可以解决哪些问题？

答案来自课件+百度

答：

- 1) 交互模型 - 交互模型处理分布式系统中性能以及设置时间限制的困难，例如对于消息传递。反映了进程交互的方式；
- 2) 故障模型 - 故障模型试图给出由进程和通信通道呈现出来的故障的一个精确的规格说明。它定义可靠的通信和正确的进程；

3) 安全模型 - 安全模型讨论对于进程和通信通道可能存在的威胁，它引入了安全通道的概念，以低于这些威胁。



三. 进程间通信（共 5 题）

0. 什么是操作系统

答：

- 1.是计算机系统中的一个系统软件，硬件之上的第一层软件
- 2.组织和管理计算机的软硬件资源，组织计算机的工作流程，控制程序的执行并向用户提供各种服务功能
- 3.五大功能：处理机管理（进程调度通信），主存管理（存储器管理，对存储空间的管理主要是对主存的管理），设备管理（I/O），文件管理，接口（为用户提供相关的界面）（作业管理）

1. 线程与进程的区别？

答：

线程是指进程内的一个执行单元，也是进程内的可调度实体。

与进程的区别：

- (1) 地址空间：进程内的一个执行单元； 进程至少有一个线程； 它们共享进程的地址空间； 而进程有自己独立的地址空间；
- (2) 资源拥有：进程是资源分配和拥有的单位， 同一个进程内的线程共享进程的资源
- (3) 线程是处理器调度的基本单位， 但进程不是。
- (4) 二者均可并发执行。

进程和线程都是由操作系统所体会的程序运行的基本单元，系统利用该基本单元实现系统对应用的并发性

进程和线程的区别在于：

简而言之，一个程序至少有一个进程，一个进程至少有一个线程。

线程的划分尺度小于进程，使得多线程程序的并发性高。

另外，进程在执行过程中拥有独立的内存单元，而多个线程共享内存，从而极大地提高了程序的运行效率。

线程在执行过程中与进程还是有区别的。每个独立的线程有一个程序运行的入口、顺序执行序列和程序的出口。但是线程不能够独立执行，必须依存在应用程序中，由应用程序提供多个线程执行控制。从逻辑角度来看，多线程的意义在于一个应用程序中，有多个执行部分可以同时执行。但操作系统并没有将多个线程看做多个独立的应用，来实现进程的调度和管理以及资源分配。这就是进程和线程的重要区别。

进程是具有一定独立功能的程序关于某个数据集合上的一次运行活动，进程是系统进行资源分配和调度的一个独立单位。

线程是进程的一个实体，是 CPU 调度和分派的基本单位，它是比进程更小的能独立运行的基本单位。线程自己基本上不拥有系统资源，只拥有一点在运行中必不可少的资源 (如程序计数器， 一组寄存器和栈)， 但是它可与同属一个进程的其他的线程共享进程所拥有的全部资源。

一个线程可以创建和撤销另一个线程；同一个进程中的多个线程之间可以并发执行。

2. 什么是进程、线程的并发控制与调度？什么是多线程并发控制？

答案来自百度

答：

进程调度：操作系统管理了系统有限资源，当有多个进程（或多个进程发出的请求）要使用这些资源时，因为资源有限性，必须按照一定的原则选择进程（请求）来占用资源，这就是调度。

线程调度：指按照特定机制为多个线程分配 CPU 的使用权。

多线程并发控制：是指从软件或者硬件上实现多个线程并发执行的技术。具有多线程能力的计算机因有硬件支持而能够在同一时间执行多于一个线程，进而提升整体处理性能。

3. 什么是进程间通信？什么是管道通信、消息队列、信号量机制、共享内存？给出定义并查阅资料举例说明。

答案来自百度

答：

进程间通信：指进程间的信息交换，其所交换的信息量，少则只是一个状态或一个数值，多则可能是成千上万个字节。

管道通信：即发送进程以字符流形式将大量数据送入管道，接收进程可从管道接收数据，二者利用管道进行通信。

消息队列：是在消息的传输过程中保存消息的容器。

信号量机制：即利用 PV 操作来对信号量进行处理。

共享内存：在多处理器的计算机系统中，可以被不同中央处理器（CPU）访问的大容量内存。

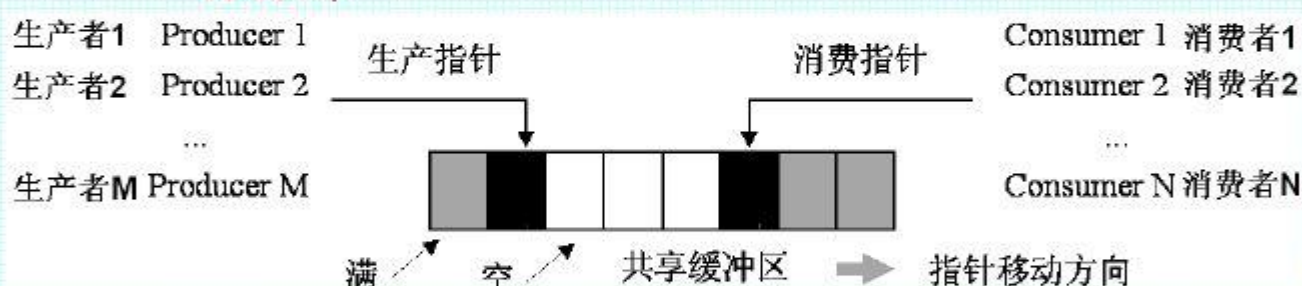
4. 利用线程通信机制的系统调用，实现操作系统经典的“生产者-消费者”RPC 问题的程序设计，要求写出详细的系统调用。

答：

进程通信经典问题

■ 问题描述（生产者/消费者问题）

- 若干进程通过无限/有限的共享缓冲区交换数据
 - 一组“生产者”进程不断写入
 - 另一组“消费者”进程不断读出
 - 共享缓冲区无限/共有N个
 - 任何时刻只能有一个进程可对共享缓冲区进行操作



生产者/消费者问题

```
semaphore n=0; /*缓冲区中  
的产品数*/
```

```
semaphore s=1; /*互斥*/
```

```
void producer() {
```

```
    while (true) {
```

```
        produce();
```

```
        semWait(s);
```

```
        append();
```

```
        semSignal(s);
```

```
        semSignal(n);
```

```
    }
```

```
}
```

```
void consumer() {
```

```
    while (true) {
```

```
        semWait(n);
```

```
        semWait(s);
```

```
        take();
```

```
        semSignal(s);
```

```
        consume();
```

```
    }
```

```
}
```

```
void main() {
```

```
    parbegin(producer,
```

```
            consumer);
```

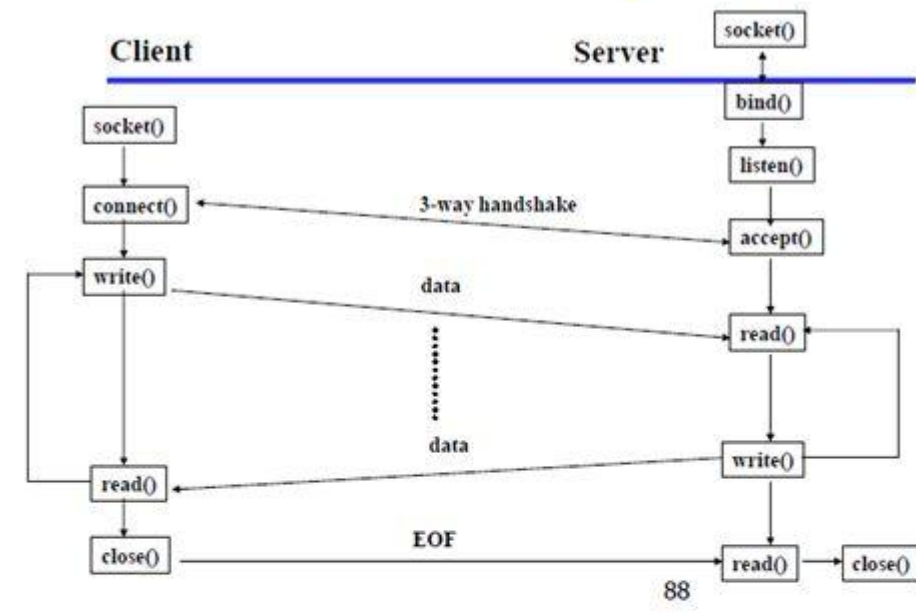
```
}
```

有限缓冲区的生产者/消费者问题

5. 什么是客户/服务器编程？如何进行客户端编程设计？如何进行服务器端编程设计？如何设计服务器与客户的通信？给出基于 Socket 的客户/服务器，面向 UDP 和 TCP 网络编程的详细说明，同时给出一个 UNIX/Linux 的 Client/Sever 的程序设计算法流程图。

答：

TCP Connection Sequence



服务端进程:

- (首先启动服务器, 过一些时间客户启动, 并连接到服务器)
- (1)使用socket调用得到一个描述符,
- (2)使用bind调用将一个名字与socket描述符连接起来, 对于Internet域就是将Internet地址联编到socket。
- (3)服务端使用listen调用指出等待服务请求队列的长度。
- (4)使用accept调用等待客户端发起连接, 一旦有客户端发出连接, accept返回客户的地址信息, 并返回一个新的socket描述符, 该描述符与原先的socket有相同的特性,
- (5)服务端使用新的socket进行读写操作。
- 说明: 一般服务端可能在accept返回后创建一个新的进程进行与客户的通信, 父进程则再到accept调用处等待另一个连接。

客户端进程:

- (1)使用socket调用得到一个socket描述符
- (2)使用connect向指定的服务器上的指定端口发起连接, 一旦连接成功返回, 就说明已经建立了与服务器的连接
- (3)通过socket描述符进行读写操作。

管套调用类型 (无连接协议UDP)



四. 操作系统支持（共 5 题）

1. 什么叫虚拟机？什么叫虚拟化？举例说明。

答案来自百度

答：

虚拟：通过某种技术把一个物理实体变成若干个逻辑上的对应物。物理实体是实际存在的，是实的，逻辑物体是用户感受到的，是虚的。

比如：在单 CPU 多道分时系统中，通过多道程序技术和分时技术把一个物理 CPU 虚拟为多台逻辑上的 CPU，使每个终端用户都认为有一台独立的 CPU 为它运行，用户感觉的 CPU 是虚拟 CPU。

虚拟机：指通过软件模拟的具有完整硬件系统功能的、运行在一个完全隔离环境中的完整计算机系统。

虚拟化：就是将事物从一种形式转变成另一种形式，最常用的虚拟化技术有操作系统中内存的虚拟化，实际运行时用户需要的内存空间可能远远大于物理机器的内存大小，利用内存的虚拟化技术，用户可以将一部分硬盘虚拟化为内存，而这对用户是透明的。又如，可以利用虚拟专用网技术（VPN）在公共网络中虚拟化一条安全、稳定的“隧道”，用户感觉像是使用私有网络一样。

2. 什么叫保护？举例说明。

3. 什么叫进程调度？什么叫线程调度？多线程并发处理？

答案来自百度

答：

进程调度：无论是在批处理系统还是分时系统中，用户进程数一般都多于处理机数、这将导致它们互相争夺处理机。另外，系统进程也同样需要使用处理机。这就要求进程调度程序按一定的策略，动态地把处理机分配给处于就绪队列中的某一个进程，以使之执行。

线程调度：计算机通常只有一个 CPU，在任意时刻只能执行一条机器指令，每个线程只有获得 CPU 的使用权才能执行指令。所谓多线程的并发运行，其实是指从宏观上看，各个线程轮流获得 CPU 的使用权，分别执行各自的任务。在运行池中，会有多个处于就绪状态的线程在等待 CPU，JAVA 虚拟机的一项任务就是负责线程的调度，线程调度是指按照特定机制为多个线程分配 CPU 的使用权。

多线程并发处理：指从软件或者硬件上实现多个线程并发执行的技术。具有多线程能力的计算机因有硬件支持而能够在同一时间执行多于一个线程，进而提升整体处理性能。具有这种能力的系统包括对称多处理机、多核心处理器以及芯片级多处理（Chip-level multithreading）或同时多线程（Simultaneous multithreading）处理器。在一个程序中，这些独立运行的程序片段叫作“线程”（Thread），利用它编程的概念就叫作“多线程处理（Multithreading）”。具有多线程能力的计算机因有硬件支持而能够在同一时间执行多于一个线程（台湾译作“执行绪”），进而提升整体处理性能。

4. 什么叫临界区？什么叫临界资源？如何解释线程同步？

答案来自百度

答：

临界区：指的是一个访问共用资源（例如：共用设备或是共用存储器）的程序片段，而这些共用资源又无法同时被多个线程访问的特性。当有线程进入临界区段时，其他线程或是进程必须等待（例如：bounded waiting 等待法），有一些同步的机制必须在临界区段的进入点与离开点实现，以确保这些共用资源是被互斥获得使用。

临界资源：一次仅允许一个进程使用的资源称为临界资源。

线程同步：在一般情况下，创建一个线程是不能提高程序的执行效率的，所以要创建多个线程。但是多个线程同时运行的时候可能调用线程函数，在多个线程同时对同一个内存地址进行写入，由于 CPU 时间调度上的问题，写入数据会被多次的覆盖，所以要使线程同步。线程同步即当有一个线程在对内存进行操作时，其他线程都不可以对这个内存地址进行操作，直到该线程完成操作，其他线程才能对该内存地址进行操作，而其他线程又

处于等待状态，目前实现线程同步的方法有很多，临界区对象就是其中一种。1) 线程同步就是线程排队。同步就是排队。线程同步的目的就是避免线程“同步”执行。

2) 只有共享资源的读写访问才需要同步。如果不是共享资源，那么就根本没有同步的必要。

3) 只有“变量”才需要同步访问。如果共享的资源是固定不变的，那么就相当于“常量”，线程同时读取常量也不需要同步。

4) 多个线程访问共享资源的代码有可能是同一份代码，也有可能是不同的代码；论是否执行同一份代码，只要这些线程的代码访问同一份可变的共享资源，这些线程之间就需要同步。

5. 影响远程调用的哪些因素会影响消息传递？

答：

调用性能是一个非常关键的因素。如果在地址空间之间分离的功能余额越多，就必须越多的使用远程调用。

远程调用涉及网络活动和可能的文件访问，在进程运行速度达到几千兆时，需要几十亿条指令周期。



五. 同步化（共 6 题）

1. 什么是物理时钟的同步化算法？

答案来自百度

答：

当两个设备一起工作并对时间有精确要求的时候，就需要在他们之间进行同步。同步是基于在两个设备之间规定一个共同的时间参考。如果这两个设备没有进行同步，无论它们开始的时间多么一致，也会由于两台设备机械结构的差异而产生时间漂移。为了实现分布系统的时间同步，有如下几种可行算法：

1) Cristian 算法：有一个时间服务器，提供标准时钟，其他系统通过询问与它同步。

2) Berkeley 算法：时间服务器没有标准时钟，它通过定期地询问各个机器的当前时间并从中求出平均值作为当前的标准时间，然后再广播给各个机器。

3) Averaging 算法：定义一个固定的同步间隔 R，每经过 R 时刻，所有的机器广播自己的当前时钟。

2. 什么是基于逻辑时钟的同步算法

答案来自百度

答：

逻辑时钟是分布式系统的特征，要求的是系统节点进程之间的相对一致性。只有相关的进程才需要有逻辑时钟同步，同步的目的是维持时间的顺序性，除时间的基本特性外，与物理时钟之间没有通用意义上的明确的关系。

3. 如何进行一致性全局状态的检测？

4. 什么是选举算法？

在分布式进程之间做出统一的决定，（确定协调者）

比如 BULLY 算法，P 向高进程发 E 消息，如果没有反应，P 选举成功，如果 Q 响应，则 Q 接管选举并继续下去。

循环算法

5. 什么是互斥算法？

当一个进程使用某个共享资源，其他进程不允许对这资源操作

集中式算法：有一个协调者：确定那个进程可以进入临界区

令牌环算法：构造一个逻辑环，得到令牌的才可进入临界区

算法	每次进出需要的消息	进入前的延迟(按消息次数)	存在问题
集中式	3	2	协调者崩溃
分布式	$2(n-1)$	$2(n-1)$	任何一个进程崩溃
令牌环	1到 ∞	0到 $n-1$	丢失令牌, 进程崩溃

6.如何进行分布式系统的死锁处理？

答案来自百度

答：

有三种处理死锁的策略：

- 1) 预防死锁。限制请求,保证前文提到的四个死锁条件中至少有一个不能发生,从而预防死锁；
- 2) 避免死锁。如果结果状态是安全的,就将资源动态地分给进程。如果至少有一个执行序列使所有的进程都能完成运行,那么这个状态就是安全的；
- 3) 检测死锁和从死锁中恢复,允许死锁发生,然后发现并解除死锁。

死锁预防和避免采用一种悲观方法,即认为死锁会经常发生并试图阻止或避免它。虽然死锁避免策略在集中式系统中广为应用,并且有许多算法,但是在分布式系统中很少使用。这是因为在分布式系统中没有全局时钟,检查安全性状态会涉及到大量进程和资源的计算,从而引起昂贵的开销。死锁检测和恢复使用乐观方法,然而这种方法对于死锁发生频繁的应用程序可能并不有效。

死锁的预防

死锁预防算法通过限制进程的资源请求方法来预防死锁。死锁预防通常通过下列方法之一来实现。它们都基于打破四个死锁条件中的一种：

- 1) 静态分配资源。要求进程必须在开始执行前就申请它所需要的全部资源,并且只有当系统能满足进程的资源申请要求并把资源分配给进程之后,该进程才开始执行。这种策略可以预防死锁的发生是由于其破坏了“占有且等待资源”和“循环等待资源”的条件,从而系统中的所有进程必然不会发生死锁。
- 2) 按序分配资源。在系统中的每一个资源都会给出一个编号。分配资源的时候作了以下规定：任何进程在申请两个以上资源时,总是按照编号的大小顺序申请。这种策略可以预防死锁的发生是由于其破坏了“循环等待资源”的条件,从而系统中的所有进程必然不会发生死锁。
- 3) 剥夺式分配资源。当一个进程申请资源得不到满足时,可从另一个拥有这种资源的进程那里去抢夺,然后继续运行。这种策略可以预防死锁的发生是由于其破坏了“非抢夺式分配”的条件,从而系统中的所有进程必然不会发生死锁。

等待-死亡方案 (Wait-die Scheme)

该方案是基于非剥夺方法。当进程 P_i 请求的资源正被进程 P_j 占有时，只有当 P_i 的时间戳比进程 P_j 的时间戳小时，即 P_i 比 P_j 老时， P_i 才能等待。否则 P_i 被卷回（roll-back），即死亡。一个进程死亡后会释放他所占用的所有资源。在这里假定死亡的进程将带着同样的时间戳重新运行。由于具有较小时间戳的进程才等待具有较大时间戳的进程，因此很显然死锁不会发生。当进程在等待特定的资源时，不会释放资源。一旦一个进程释放一个资源，与这个资源相联系的等待队列中的一个进程将被激活。

伤害-等待方案（Wound-wait Scheme）

它是一种基于剥夺的方法。当进程 P_i 请求的资源正被进程 P_j 占有时，只有当进程 P_i 的时间戳比进程 P_j 的时间戳大时，即 P_i 比 P_j 年轻时， P_i 才能等待。否则 P_j 被卷回（roll-back），即死亡。只要被卷回的进程重新启动时，使用原有的时间戳，这两种方案都能避免死锁和饿死现象。由于时间戳总是增加的，被卷回的进程最终将具有最小的时间戳。

死锁的检测

为了降低系统开销，在分配资源时会不加限制，只要系统中有剩余的资源，总是把资源分配给申请者。显然，这样的结果是可能会出现死锁。那么，为了使系统能够正常工作，在系统中会采用定时运行一个“死锁检测”程序的方法，当检测到死锁时该程序将会设法将其排除。在分布式系统中的死锁检测法不会造成很多不必要的进程流产，但是也会增加了系统的额外开销和复杂度。

1. 集中式死锁检测

在分布式系统中，每个站点都有一个本地死锁检测程序，其任务是判断在其站点所有潜在的全局死锁；其方法是：在站点的输入端口开始，沿本地等待图反向搜索，如果最终会搜索到输出端口，就说明具有潜在的全局死锁。

2. 分布式死锁检测

分布式死锁检测和集中式的主要差别是：在集中式方案中全部潜在的死锁循环都发送给某个指定的站点，而在分布式检测方案中则没有这种站点。分布式死锁检测机构中没有本地和非本地死锁检测程序的任何区别，每个站点具有同样的责任。在分布式方案中，死锁检测程序需要一种规则来决定应该把潜在的死锁循环发送给哪个站点，这种规则必须保证能最终检测到全局死锁，并且必须尽量减小传送的信息量。

Knapp 将分布式死锁检测算法分为以下四类：

1) 路径推动算法（path-pushing algorithm）。先在每个机器上建立形式简单的全局等待图。每当进行死锁检测时，各个机器就将等待图的拷贝送往一定数量的邻居节点。局部拷贝更新后又传播下去。这一过程重复进行直到某个节点获得了足够的信息来构造一个等待图以便做出是否存在死锁的结论。不幸的是，这类算法中的许多算法在实际中是错误的。主要原因是传输过程中的部分等待图并不能代表整个全局等待图，因为各个节点采集数据的方法是异步的。

2) 边跟踪算法（edge-chasing algorithm）。分布式网络结构图中的回路可以通过沿图的边传播一种叫探测器的特殊信息来检测。当一个发起者得到一个与自己发送的探测器相匹配的探测器时，它就知道它在图中的一个回路里。

3) 扩散计算（diffusing computation）。怀疑有死锁发生时，事务管理器通过向依赖于它的进程发送查询启动一个扩散进程。这里不会生成全局等待图。发送查询信息时，扩散计算就增长；接收回答后，扩散计算就缩减。根据所得信息，发起者会检测到死锁的发生。典型情况是，扩散进程动态地生成等待图的一个子树。

4) 全局状态检测 (global state detection)。这个方法基于 Chandy 和 Lamport 的快照方法。可以通过建立一个一致的全局状态而无需暂停当前的计算来生成一个一致的全局等待图。

六. 分布式文件系统 (共 2 题)

1. 分布式文件系统的特点与需求是什么?

答案来自百度

答:

特点:

持久性 (即便程序终止, 信息仍可用), 信息共享, 缓存/副本 (可能有多个文件副本存放在不同的地方, 并被检索到) (一般有三个副本), 一致性

需求:

透明性: 访问透明性: 提供了一个单一的操作集合来访问本地和远程的文件;

位置透明性: 用户程序看到的是一个统一的文件名空间。文件或者文件组可以重新部署, 无需改变他们的路径名称

移动透明性: 当文件被移动后, 用户程序和用户节点的系统管理员表均不需要改变

☑ 性能透明性: 当服务负载在一定范围内变化时, 客户程序可以保持满负荷的性能

☑ 扩展透明性: 文件服务可以扩充, 以满足负载和网络规模的增长

并发文件更新

☑ 并发控制: 客户改变文件的操作不影响其他用户访问或改变同一文件的操作

文件复制: 多个副本

☑ 更好的性能与容错

硬件和操作系统异构性: 文件服务的接口必须有明确的定义。在不同操作系统和计算机上实现客户和服务软件

服务界面不应受限于少数一些操作系统或硬件系统

⌘ 容错

☑ 为了处理暂时的通信错误, 容错设计可以基于最多一次性语义

☑ 对于幂等操作: 支持最少一次性语义

☑ 无状态的服务器: 崩溃重启时不需恢复

⌘ 一致性

☑ Unix 提供单一副本更新语义

☑ 当文件在不同地点被复制和缓存时, 可能会偏离单一副本更新语义

安全性: 通常是通过访问控制表来实现;

效率: 分布式文件系统的性能应至少和常规的文件系统相当

2. Sun 网络文件系统 (NFS) 的自动安装器是如何改进 NFS 的性能和可伸缩性的?

七. 分布式系统前沿 (共 2 题)

1. 什么是大数据? 什么是云平台? 举例说明它们的关系。

答案来自百度

答:

大数据 - 研究机构 Gartner 给出了这样的定义。“大数据”是需要新处理模式才能具有更强的决策力、洞察发现力和流程优化能力来适应海量、高增长率和多样化的信息资产。

麦肯锡全球研究所给出的定义是：一种规模大到在获取、存储、管理、分析方面大大超出了传统数据库软件工具能力范围的数据集合，具有海量的数据规模、快速的数据流转、多样的数据类型和价值密度低四大特征。

云平台 - 提供基于“云”的服务，供开发者创建应用时采用。可以划分为 3 类：以数据存储为主的存储型云平台，以数据处理为主的计算型云平台以及计算和数据存储处理兼顾的综合云计算平台。

关系 - 从技术上看，大数据与云计算的关系就像一枚硬币的正反面一样密不可分。大数据必然无法用单台的计算机进行处理，必须采用分布式架构。它的特色在于对海量数据进行分布式数据挖掘。但它必须依托云计算的分布式处理、分布式数据库和云存储、虚拟化技术。

随着云时代的来临，大数据也吸引了越来越多的关注。分析师团队认为，大数据通常用来形容一个公司创造的大量非结构化数据和半结构化数据，这些数据在下载至关系型数据库用于分析时会花费过多时间和金钱。大数据分析常和云计算联系到一起，因为实时的大型数据集分析需要像 MapReduce 一样的框架来向数十、数百或甚至数千的电脑分配工作。

大数据需要特殊的技术，以有效地处理大量的容忍经过时间内的数据。适用于大数据的技术，包括大规模并行处理（MPP）数据库、数据挖掘、分布式文件系统、分布式数据库、云计算平台、互联网和可扩展的存储系统。

2. 什么是网格？网格平台与云平台的联系与区别？

答案来自课件

答：

网格是信息社会的网络基础设施，它把整个因特网整合成一台巨大的超级虚拟计算机，实现互联网上所有资源的互联互通，完成计算资源、存储资源、通信资源、软件资源、信息资源、知识资源等智能共享的一种新兴的技术。

联系与区别 - 云计算和网格彼此相关，但又有不同。网格计算强调资源共享，任何人都可以做为请求者使用其它节点的资源，任何人都需要贡献一定资源给其他节点。网

格计算强调将工作量转移到远程的可用计算资源上。云计算强调专有，任何人都可以获取自己的专有资源，并且这些资源是由少数团体提供的，使用者不需要贡献自己的资源。在云计算中，计算资源被转换形式去适应工作负载，它支持网格类型应用，也支持非网格环境，比如运行传统或 Web2.0 应用的三层网络架构。网格计算侧重并行的计算集中性需求，并且难以自动扩展。云计算侧重事务性应用，大量的单独的请求，可以实现自动或半自动的扩展。